

Examen Parcial Práctico
Brenner Ojeda
Semestre 2025-1

IMPORTANTE:

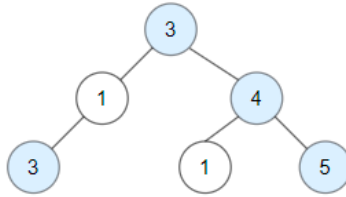
- Enviar en canvas un **único archivo .txt**
- Colocar nombres y apellidos del alumno al inicio del archivo .txt.

Ejercicios

1. (6 pts) Dado el **root** de un árbol binario, se dice que un nodo $\mathcal{X}$ es **bueno** si en el camino entre el **root** y $\mathcal{X}$ no existen nodos con valores estrictamente mayores a $\mathcal{X}$.

Diseñe un algoritmo que cuente la cantidad de nodos **buenos** en **root**.

■ **Ejemplo 1:**



Input: root = [3,1,4,3,null,1,5]

Output: 4

Explicación: Los nodos celestes son **buenos**.

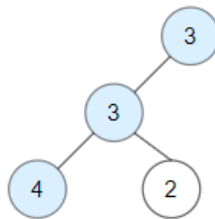
Root (3) Siempre es un nodo **bueno**.

Node 4 ->(3,4) es el valor máximo en el camino.

Node 5 ->(3,4,5) es el valor máximo en el camino.

Node 3 ->(3,1,3) es el valor máximo en el camino.

■ **Ejemplo 2:**



Input: root = [3,3,null,4,2]

Output: 3

Explicación:

Node 2 ->(3, 3, 2) no es **bueno**, porque 3 es mayor que él.

Restricciones:

- La cantidad de nodos en el árbol está entre $[1, 10^5]$

- Cada nodo $\mathcal{X}$ puede tomar valores entre $[-10^4, 10^4]$

Sigue el template del problema para ser llenado. No debe modificar la plantilla excenpto donde dice 'PUT YOUR CODE'. Solo debe de colocar su código en el método **goodNodes**.

```
#include <cassert>
#include <iostream>
#include <cmath>
#include <algorithm>
#include <iostream>
using namespace std;

// Definition for a binary tree node.
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};

class Solution {
public:
    int goodNodes(TreeNode* root) {
        ////////////////
        //PUT YOUR CODE//
        ////////////////
    }
};

int main() {
    Solution sol;
    // Test 1: Single node
    TreeNode* test1 = new TreeNode(5);
    assert(sol.goodNodes(test1) == 1);

    // Test 2: Tree with some good nodes
    TreeNode* test2 = new TreeNode(3);
    test2->left = new TreeNode(1);
    test2->left->left = new TreeNode(3);
    test2->right = new TreeNode(4);
    test2->right->right = new TreeNode(5);
    assert(sol.goodNodes(test2) == 4);

    // Test 3: Árbol con todos los nodos iguales
    TreeNode* test3 = new TreeNode(2);
    test3->left = new TreeNode(2);
    test3->right = new TreeNode(2);
    test3->left->left = new TreeNode(2);
    test3->left->right = new TreeNode(2);
    assert(sol.goodNodes(test3) == 5); // todos son "good nodes"

    // Test 4: Árbol creciente en línea recta (derecha)
    TreeNode* test4 = new TreeNode(1);
```

```

test4->right = new TreeNode(2);
test4->right->right = new TreeNode(3);
test4->right->right->right = new TreeNode(4);
assert(sol.goodNodes(test4) == 4); // cada nodo es mayor que el anterior

// Test 5: Árbol decreciente
TreeNode* test5 = new TreeNode(5);
test5->left = new TreeNode(3);
test5->left->left = new TreeNode(1);
test5->right = new TreeNode(4);
assert(sol.goodNodes(test5) == 1); // solo la raíz

// Test 6: Árbol donde cada hijo izquierdo es mayor que el padre
TreeNode* test6 = new TreeNode(1);
test6->left = new TreeNode(2);
test6->left->left = new TreeNode(3);
assert(sol.goodNodes(test6) == 3); // todos son good nodes

// Test 7: Árbol zig-zag con algunos valores menores
TreeNode* test7 = new TreeNode(3);
test7->right = new TreeNode(1);
test7->right->left = new TreeNode(4);
assert(sol.goodNodes(test7) == 2); // 3 y 4 son good nodes

// Test 8: Árbol con valores negativos
TreeNode* test8 = new TreeNode(-1);
test8->left = new TreeNode(-2);
test8->right = new TreeNode(-3);
assert(sol.goodNodes(test8) == 1); // solo -1 es good node

// Test 9: Árbol más complejo con mezcla de buenos y no buenos nodos
TreeNode* test9 = new TreeNode(5);
test9->left = new TreeNode(1);
test9->right = new TreeNode(7);
test9->right->left = new TreeNode(6);
test9->right->right = new TreeNode(9);
assert(sol.goodNodes(test9) == 3); // 5, 7, 9

// Test 10: Árbol vacío
TreeNode* test10 = nullptr;
assert(sol.goodNodes(test10) == 0);
std::cout << "All tests passed!\n";

return 0;
}

```

Si el código pasa todos los test:



```

input
All tests passed!

...Program finished with exit code 0
Press ENTER to exit console.

```